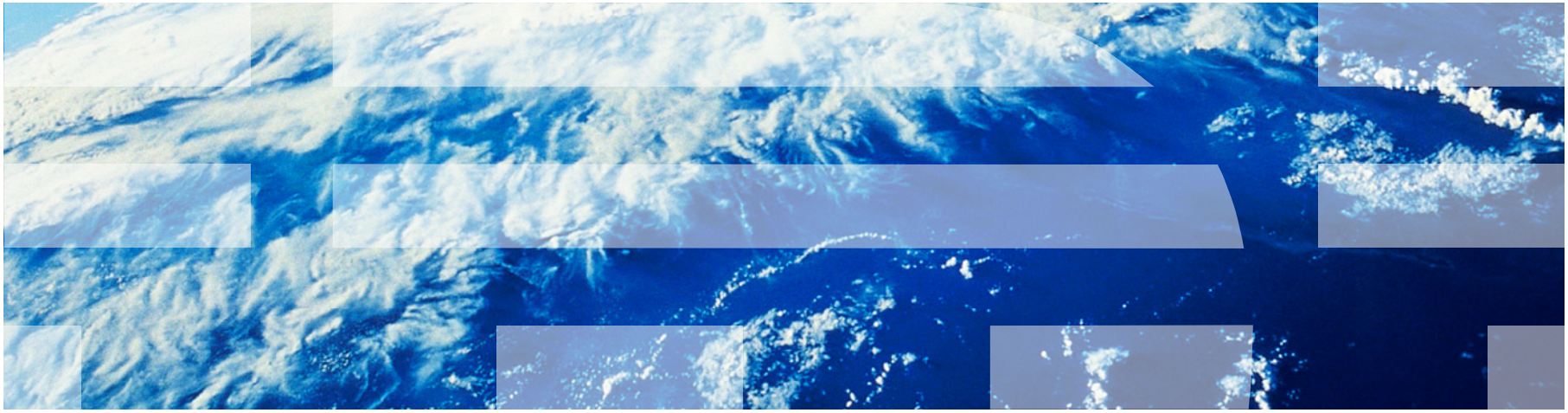
JOINs and Aggregations



Trade off between table complexity and query complexity

Students(cuid: *string*, name: *string*, gpa: *float*)

Enrolled(student_id: *string*, cid: *string*, grade: *string*)

- What is the GPA of all students enrolled in CSEE 4121?

- A possible (cumbersome solution) → create a new franken-table
  - A single attribute for each possible class:

FrankenTable(student_id: *string*, grade_course1: *string*, *grade*_course2: *string*, *…)*

  - Hundreds of attributes, most columns are NULL

Joins

Product(PName, Price, Category, Manufacturer)
Company(CName, StockPrice, Country)

Ex: Find all products under $200 manufactured in Japan; return product names and prices.

```
SELECT PName, Price
FROM   Product, Company
WHERE  Manufacturer = CName
       AND Country='Japan'
         AND Price <= 200
```

A **join** between tables returns all unique combinations of their tuples **which meet some specified join condition**

Joins

Product

PName	Price	Category	Manufacturer
Gizmo	$19	Gadgets	GizmoWorks
Powergizmo	$29	Gadgets	GizmoWorks
SingleTouch	$149	Photography	Canon
MultiTouch	$203	Household	Hitachi

Company

CName	Stock Price	Country
GizmoWorks	25	USA
Canon	65	Japan
Hitachi	15	Japan

```
SELECT PName, Price
FROM   Product, Company
WHERE  Manufacturer = CName
       AND Country='Japan'
           AND Price <= 200
```

PName	Price
SingleTouch	$149

An example of SQL semantics

```
SELECT R.A
FROM   R, S
WHERE  R.A = S.B
```

Input

R

A
1
3

S

B	C
2	3
3	4
3	5

Cross Product

A	B	C
1	2	3
1	3	4
1	3	5
3	2	3
3	3	4
3	3	5

Apply Selections / Conditions

Apply Projection

A	B	C
3	3	4
3	3	5

Output

A
3
3

Note: this is how SQL logically works, not actually how it's implemented

- The preceding slide show *what a join means*
- Not actually how the database executes it under the covers